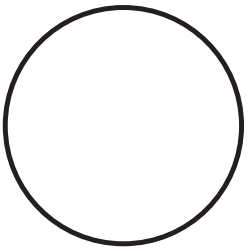
5

DIFFERENTIAL EQUATIONS AND VECTORS



The physicist Richard Feynman once quipped that calculus is the language God talks. If that's true, then the divine dialogue is written in *differential equations*. In this chapter, we'll formalize differential equations before solving them numerically. With that foundation in place, I'll introduce vectors as helpful tools for writing differential equations involving motion. Then I'll show you how to perform vector arithmetic using the powerful NumPy package. By the end of the chapter, you'll get your first glimpse of chaos.

Differential equations excel at describing motion, and it's natural to think of motion using arrows. For example, we can point to a `Turtle` object's location and point in the direction it's moving (velocity). With a small conceptual leap powered by calculus, we can also point in the direction its velocity changes, perhaps due to an invisible pull. In the real world, I wrote much of this book in a tea shop that raises chickens in their garden out back. The

chickens are fun to watch and Figure 5-1 shows their characteristic flight pattern, which is basically a hop.

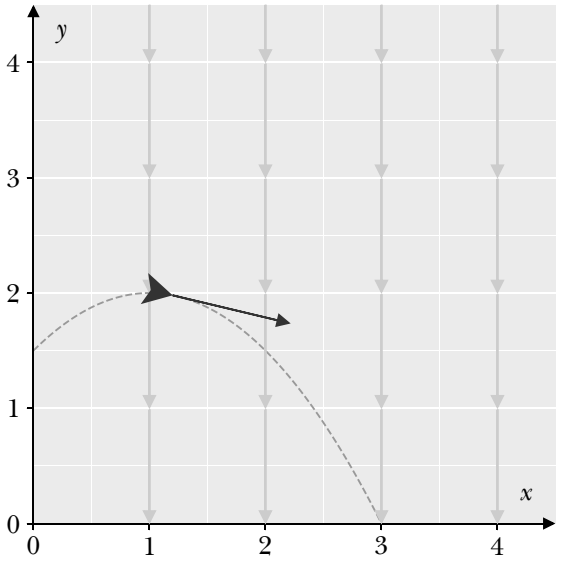


Figure 5-1: A chicken following its velocity vector

After climbing to its perch at (0, 1.5), the chicken hopped up and out, destined to follow the medium gray, dashed parabola. The diagram captures the chicken's motion in mid flight. The dark gray arrow extending from its tip represents its velocity. This arrow extends 1 unit to the right and 0.2 units down, which we can write compactly as (1, −0.2). The light gray arrows pointing downward represent the invisible pull of gravity, which we can write compactly as (0, −1). We'll spend this chapter formalizing arrows as *vectors* that support arithmetic like addition and scaling. In code, we could imagine using a data structure that's similar to a list to represent each vector:

```
pos = [0.0, 1.5]
vel = [1.0, 1.0]
acc = [0.0, -1.0] # Gravity!
```

After having a `Turtle` object teleport to its starting position, `pos`, we could steadily integrate forward in time, first using `acc` to update `vel` and then using `vel` to update `pos`. The code would look something like this:

```
# Make the Turtle t hop to the ground
dt = 0.001
while t.ycor() > 0:
    # Update velocity
    vel += dt * acc
    # Update position
    pos += dt * vel
```

```
    # Set heading toward next position
    x, y = pos
    angle = t.towards(x, y)
    t.seth(angle)
    # Glide to the next point
    t.goto(pos)
```

Recall from Chapter 3 that acceleration is the derivative of velocity and velocity is the derivative of position. We express changes to these values using *differential equations*, which are difference equations taken to the limit. Let's go ahead and formalize this idea.

First, create a new notebook file called *chapter_05.ipynb* in which to enter the code examples for this chapter. Be sure to download a copy of your notebook and module when you want to save your progress.

Solving Ordinary Differential Equations

An *ordinary differential equation* expresses how a function changes with respect to a single variable. For example, it might capture how the size of a population changes over time. A *partial differential equation* expresses how a function changes with respect to more than one variable. For example, atmospheric temperature changes throughout time and space. In this chapter, we'll cover two methods of solving ordinary differential equations: separation of variables and Euler's method.

Separation of Variables

After reading this chapter, you may feel compelled to launch self-replicating robots into space so that they can explore the cosmos. Let's call these robots "germinators." To plan their journey, you'll need to predict how the germinator population will change over time. Given a rule for robot replication, it'd be helpful to find a function, g, that tells us the population size at a given time, t. We call such function a *solution* to the differential equation.

Here is a simple ordinary differential equation that describes this scenario:

$$\dot{g} = kg$$

This differential equation says that the rate at which a value g changes is proportional to its current value. The letter k, which represents the growth rate, is called a *parameter* in the context of differential equations. k isn't a variable like time t or population size g. Instead, it's a value that we tweak to make the model work. For example, the differential equations $\dot{g} = 2g$ and $\dot{g} = 5g$ describe different population scenarios, but the basic dynamics are the same: The population size changes based on its current value.

In our self-replicating robot scenario, each germinator copies itself, so the rate at which the number of germinators changes is *directly proportional* to the current number of germinators. That means a few robots can build

only a few robots and many robots can build many robots. We can express this mathematically like so:

$$\dot{g} \propto g$$

The symbol $\propto$ means "proportional to" in this context. The differential equation $\dot{g} = kg$ turns the high-level description of this relationship into a model we can apply.

The simple model $\dot{g} = kg$ has an *explicit solution*, meaning we can write an equation for the function g using the independent variable t. Let's use a technique called *separation of variables* to find the solution. First, we rewrite the equation using $\frac{dg}{dt}$ to express the derivative:

$$\frac{dg}{dt} = kg$$

We then move all the terms with the variable g to the left and all the terms with the variable t, and any leftover terms, to the right, thereby separating them:

$$\frac{dg}{g} = dt\, k$$

Separation of variables may seem like a sleight of hand at first, but rest assured that it's a sound technique. You could prove that it works by applying the chain rule if you're curious and careful.

Once we've separated the variables, we integrate each side with respect to its variable:

$$\int dg \frac{1}{g} = \int dt\, k$$

We got lucky because we can express these antiderivatives in closed form. On the left, we get $\ln |g| + C_g$, and on the right, we get $kt + C_t$. The constants of integration C_g and C_t are still just placeholders for specific numbers. After writing the antiderivatives, we steadily work our way toward an equation for g:

$$\begin{aligned}
\ln |g| + C_g &= kt + C_t \\
\ln |g| &= kt + C_t - C_g \\
&= kt + C \\
|g| &= e^{kt+C} \\
&= e^C e^{kt} \\
&= g_0 e^{kt}
\end{aligned}$$

$$g(t) = g_0 e^{kt}$$

In the final equation, the term g_0 is the initial germinator population, which doesn't vary over time. The solution is identical to the exponential growth/decay model we first met in Chapter 3. This makes sense given that the robot population literally builds on itself.

Let's confirm that the function $g(t) = g_0 e^{kt}$ actually solves the differential equation $\dot{g} = kg$:

$$\begin{aligned}
\dot{g} &= \frac{d}{dt} g \\
&= \frac{d}{dt}(g_0 e^{kt}) \\
&= g_0 \frac{d}{dt} e^{kt} \\
&= g_0 k e^{kt} \\
&= k(g_0 e^{kt}) \\
&= kg
\end{aligned}$$

Houston, we have a solution!

As we've discussed, most antiderivatives, and hence most differential equations, don't behave as nicely. We simply can't write most explicit solutions in closed form. All is not lost, though, because we can estimate solutions using a familiar approach.

Euler's Method

The 2016 film *Hidden Figures* tells the story of the human "calculators" at NASA who helped to launch the first Americans into space. In one scene, a groundbreaking mathematician named Katherine Johnson (Figure 5-2) realizes that she can use estimation to solve an otherwise unsolvable problem. The issue, she says,

> is when the capsule moves from an elliptical orbit to a parabolic orbit. There's no mathematical formula for that. We can calculate launch, landing, but without this conversion, the capsule stays in orbit; we can't bring it home... Maybe it's old math. Something that looks at the problem numerically and not theoretically.

The "old math" Johnson and her colleagues apply using early digital computers is called *Euler's method*.

Figure 5-2: Katherine Johnson working at NASA circa 1966

We've actually been applying Euler's method in disguise throughout the last couple of chapters. It's a deceptively simple idea given to us by Leonhard Euler, who made a dizzying array of contributions to mathematics and physics. Given a differential equation and a starting point, or *initial value*, Euler's method lets us estimate the next output of a function. For example, let's say that the initial robot population is g_0 and the starting time is t_0. We can use Euler's method to estimate the future population like so:

$$g_1 = g_0 + h\dot{g}(t_0)$$

The value h is called the *step size*, and it represents the amount of time between estimates. Smaller time steps generally produce better estimates. The trade-off for improving an estimate is increased work. For example, a time step that's twice as small requires twice as many computations.

We can use Euler's method to estimate the future germinator population by performing a series of estimates like so:

$$\begin{aligned} g_1 &= g_0 + h\dot{g}(t_0) \\ g_2 &= g_1 + h\dot{g}(t_1) \\ &\dots \\ g_N &= g_{N-1} + h\dot{g}(t_{N-1}) \end{aligned}$$

We compute g_1 and then use the resulting g_1 value to compute g_2, and so on, until we've come to the end of our interval. More concisely, we perform the following summation:

$$g_N = g_0 + \sum_{i=0}^{N-1} h\dot{g}(t_i)$$

Seem familiar? Euler's method looks nearly identical to computing a Riemann sum, and you may recognize h as δ from the forward finite difference equation. The big difference between these techniques is their purpose. A Riemann sum estimates the area under a curve. This is a general idea, and it doesn't rely on the concept of a differential equation at all. Euler's method, on the other hand, estimates the solution to differential equations. Over the course of the chapter, you'll see that it's often helpful to think of such solutions as paths rather than areas. You'll also see that it's possible to apply Euler's method to differential equations that include higher derivatives.

Now that we're better acquainted with Euler's method, let's apply it to the problem of calculating the self-replicating robots' population growth. Say we start with one robot that can copy itself in one week. Without doing any calculus, we know that the function $g(t) = 1 \cdot 2^t$ describes the growth of the robot population because the population doubles each week. Table 5-1 tracks the robot population as it grows over a few weeks.

Table 5-1: Robot Population

Weeks	Population
0	1
1	2
2	4
3	8

Since we already went through the trouble of modeling this scenario using a differential equation, let's express the growth model along with its initial conditions:

$$\begin{aligned} g_0 &= 1 \\ k &= \ln(2) \\ g(t) &= g_0 e^{kt} \end{aligned}$$

You'll show that these two models are equivalent in an exercise.

Figure 5-3 shows how the population would grow over the course of a couple of months according to our explicit solution and to estimates made with Euler's method.

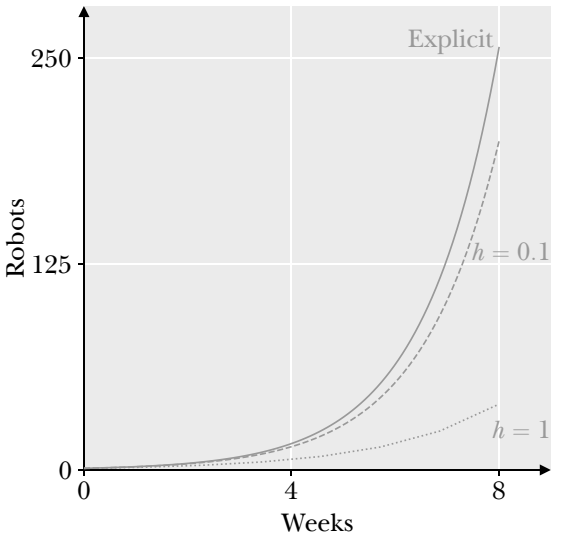


Figure 5-3: Estimating solutions with different step sizes

The graph shows the explicit solution curve in solid gray, along with two numerical solutions in dashed and dotted line styles. The explicit solution we obtained through separation of variables says that $g(8) = 1 \cdot e^{(\ln(2))\cdot 8} = 256$ robots.

With a large step size of 1, Euler's method projects the robot population to be about 40 after eight weeks. That's a severe underestimate. Reducing the step size to 0.1 brings the projection to about 200 robots after eight weeks, which is better. Reducing the step size to 0.001 would bring that population projection to 255 robots, which is pretty close!

Higher-Order Differential Equations

Euler's method applies only to *first-order* differential equations, meaning the highest derivative included is a first derivative. Our self-replicating population model is a good example:

$$\dot{g} = kg$$

Many important differential equations are *second-order*, meaning they include a second derivative. Consider Newton's second law of motion:

$$\ddot{x} = \frac{1}{m}F$$

This law is a second-order ordinary differential equation because acceleration, $\ddot{x}$, is the second derivative of position. According to this law, a force F causes a body to accelerate, which changes its velocity, which in turn changes its position.

With a slight modification, we can use Euler's method to estimate solutions to higher-order differential equations. The trick is to rewrite a higher-order equation as multiple first-order equations. In the case of Newton's second law, we'll express acceleration as the first derivative of velocity $\dot{v}$ and not the second derivative of position $\ddot{x}$.

$$\dot{v} = \frac{1}{m}F$$

With this slight change in perspective, Euler's method can easily produce estimates for future velocities. All we need are initial values t_0, x_0, and v_0. Let's begin by estimating velocity:

$$\begin{aligned} v_1 &= v_0 + h\dot{v}(t_0) \\ v_2 &= v_1 + h\dot{v}(t_1) \\ &\dots \\ v_N &= v_{N-1} + h\dot{v}(t_{N-1}) \end{aligned}$$

We compute the derivative of velocity at each time step $\dot{v}$ based on the forces being applied at that moment. Because we can compute a body's velocity at each time step, we can use velocity to estimate its future positions. Here's how we organized the computation previously:

$$\begin{aligned} x_1 &= x_0 + h\dot{x}(t_0) \\ x_2 &= x_1 + h\dot{x}(t_1) \\ &\dots \\ x_N &= x_{N-1} + h\dot{x}(t_{N-1}) \end{aligned}$$

Keep in mind that velocity is the first derivative of position, or $\dot{x}(t_i) = v_i$. That means we could rewrite the computation as follows:

$$\begin{aligned} x_1 &= x_0 + hv_0 \\ x_2 &= x_1 + hv_1 \\ &\dots \\ x_N &= x_{N-1} + hv_{N-1} \end{aligned}$$

Those v_i terms all come from applying Euler's method, as we discussed a moment ago. During each time step, we estimate the body's next velocity, and then we use that velocity to estimate the body's next position. The smaller the time step, the better the estimate.

Now that you know how to solve first- and second-order differential equations with Euler's method, let's put the technique to use.

Euler's Method in Python

Applying Euler's method in Python is similar to computing a Riemann sum. Let's write code that can estimate the robot population growth function we've discussed thus far. Open *chapter_05.ipynb* and enter the following code:

```
import math

# Set the initial values
t_0 = 0
g_0 = 1

# Set the parameter
k = math.log(2)

# Configure the time step
t_f = 8
dt = 0.1
num_steps = int(t_f / dt)

# Run Euler integration
g = g_0

for i in range(1, num_steps):
    # Compute the change in population
    g_change = dt * k * g
    # Compute the population for the next time step
    g_next = g + g_change
    # Update the population variable
    g = g_next

# Print the final state
print(g)
--------
199.2223300968443
```

First, we import the `math` module from the Python standard library. Next, we begin the simulation by setting the initial values for time and the size of the robot population. Then we set the growth parameter k to $\ln(2)$ so that the population doubles each week and configure the time settings, including the final time, step size, and number of steps needed. Once we've fully configured the simulation, we begin integrating through time using Euler's method.

Before we start looping, we use the statement `g = g_0`, which sets the current population size to its initial value. Notice that the loop begins counting up from `1` instead of the default `0` to account for the initial value. Within the loop, we use the differential equation $\dot{g} = kg$ to compute the population change, compute the next value of the population size, and finally update

the population before the next time step. Once we finish looping, we print the estimated robot population after eight weeks.

After all that, our estimate is off by more than 50 robots. Oops. Let's decrease our step size to 0.001 and try again:

```
--snip--

# Configure the time step
t_f = 8
dt = 0.001 # Tiny!
num_steps = int(t_f / dt)

--snip--

# Print the final state
print(g)
--------
255.33173288111493
```

Now our estimate is only off by half a robot!

As we've seen, applying Euler's method with a small step size can produce a good estimate of the solution to a differential equation. Yet a smaller step size means more computation, which in turn means consuming more time and energy. We'll continue analyzing the performance of our algorithms in the coming chapters. In the meantime, we'll supercharge differential equations using a new mathematical object.

Vector Arithmetic

In mathematics, a *vector* is an ordered list of values. For example, we could use vectors to represent a `Turtle`'s position (x, y) and velocity $(\dot{x}, \dot{y})$ in 2D space. In this section, we'll develop basic vector arithmetic and use it to express 2D motion with differential equations.

Different Views of Vectors

As with most good ideas, there's more than one way to view a vector. Algebraically, a vector is an ordered list of values, called its *components*. For example, a `Turtle`'s position vector $\mathbf{p} = (1, 2)$ has the components 1 and 2, which correspond to the `Turtle`'s x- and y-coordinates. Vectors are commonly written in boldface, as in $\mathbf{p}$, or with an arrow, as in $\vec{p}$. We'll use boldface throughout the book.

Geometrically, we can think of the vector $\mathbf{p}$ as the point at $(1, 2)$, or as an arrow extending from the origin to that point. Figure 5-4 demonstrates both geometric points of view.

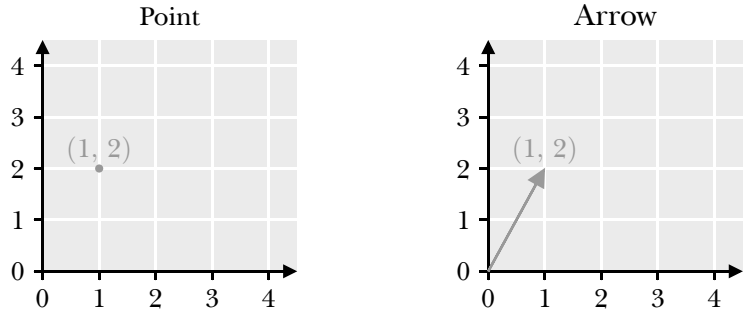


Figure 5-4: Two views of vectors

The diagram on the left plots a point with coordinates (1, 2). The diagram on the right depicts an arrow pointing from the origin to the point (1, 2). Note that both diagrams use coordinates in space to represent the vector's components. As we'll see, both views are helpful in different contexts.

A vector's individual components are *scalar* values, meaning we only consider their position relative to zero on the number line. Real numbers are all scalar values. By contrast, a vector has magnitude (its length) and direction. Figure 5-5 labels the magnitude and direction of the vector **p**.

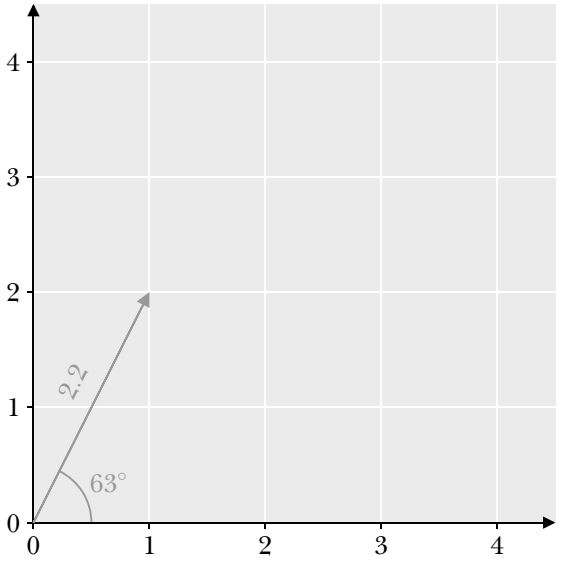


Figure 5-5: Vector magnitude and direction

The vector reaches from the origin to the point (1, 2). According to Pythagoras's theorem, its magnitude is $|\mathbf{p}| = \sqrt{1^2 + 2^2} \approx 2.2$ units. The vertical lines mean "magnitude" in this context. Given the vector's components, we can use a little trigonometry to find its direction as $\tan^{-1}\left(\frac{2}{1}\right) \approx 63°$ relative to the positive x-axis. We can also compute the vector's components given its magnitude and direction:

$$\begin{aligned} x &= |\mathbf{v}| \cos(\theta) \\ &\approx 2.2 \cos(63°) \end{aligned}$$

$$\begin{aligned} y &= |\mathbf{v}| \sin(\theta) \\ &\approx 2.2 \sin(63°) \end{aligned}$$

A vector is defined by its components, not by the location at which it's drawn, so we can draw the same vector anywhere. Figure 5-6 shows the same vector drawn in two positions.

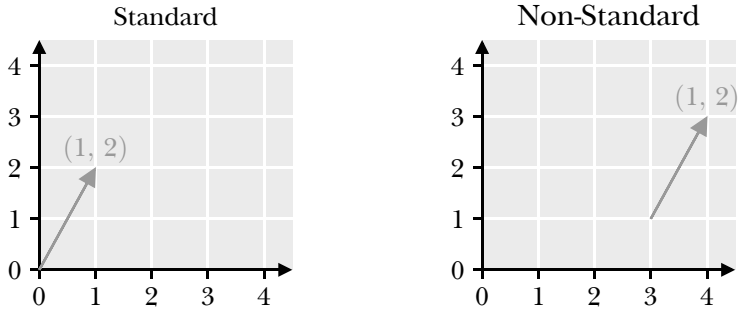


Figure 5-6: The same vector drawn in different positions

We say that a vector with its tail pinned at the origin is in *standard position*. Standard position is helpful for inspecting vectors, and for introducing the vector operations we'll use throughout the book. On the left, the vector (1, 2) is in standard position. On the right, the same vector (1, 2) is drawn from a different starting point. Both vectors have the same components, so they're equivalent.

Vectors excel at modeling where things are going. For example, the vector (0, −1) could describe an object's velocity as it falls. We'll refer to physical objects with mass as *bodies*. Figure 5-7 previews the power we gain from drawing vectors wherever they're needed.

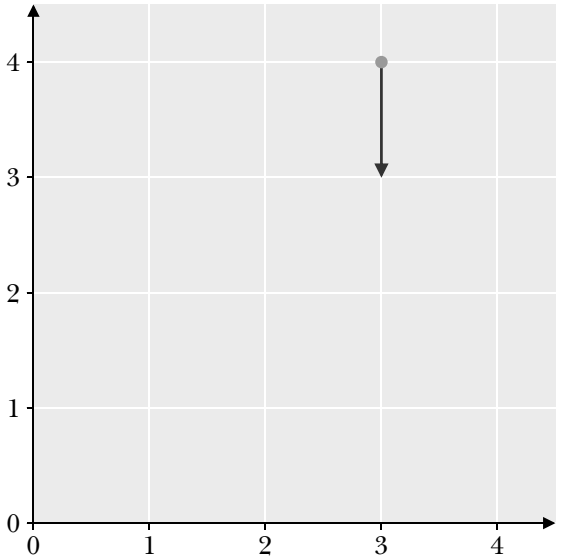


Figure 5-7: A vector pointing downward

The vector (0, −1) extends from the center of a circle centered at coordinates (3, 4). The diagram could depict a hopping chicken descending toward its landing point or an electron moving toward a positively charged surface. The visualization remains the same despite the radically different scenarios. Figure 5-8 completes this picture by describing the body's position with another vector.

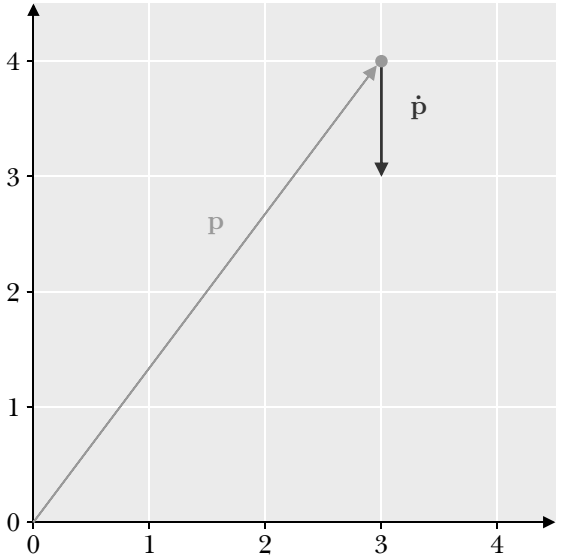


Figure 5-8: A preview of vector motion

In this view, the vector $\mathbf{p} = (3, 4)$, drawn in standard position, describes the body's current position. The vector $\dot{\mathbf{p}} = (0, -1)$ describes how that position changes over time. One arrow points to the body's current position, and another points in the direction it will move. We'll combine these vectors to predict the body's next position using Euler's method later in the chapter. To do so, we need to add a little vector arithmetic to our toolkit.

Vector Addition

We can add two vectors to produce a new vector, called the *resultant* vector. For example, imagine you measured a `Turtle`'s movement for one second and found that it traveled one step to the right and two steps up. We could describe its change in position with the vector (1, 2). During the next second, let's say the `Turtle` moved three steps to the right and one step up, represented by the vector (3, 1). Where did the `Turtle` end up after its walkabout? Geometrically, you can imagine watching the `Turtle` follow a couple of calls to `goto` with its pen down. Figure 5-9 shows that this view is equivalent to adding vectors from "tip" to "tail."

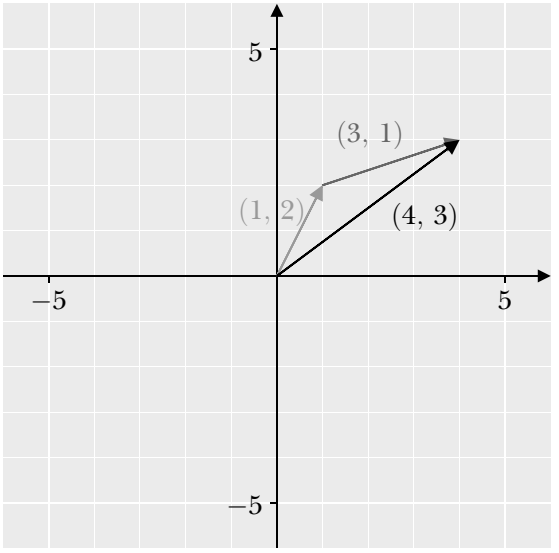


Figure 5-9: Adding two vectors

Starting from the origin, we add the vectors (1, 2), drawn in medium gray, and (3, 1), drawn in dark gray. The resultant vector, drawn in black, points directly from the beginning of the `Turtle`'s journey to its end. We can inspect Figure 5-9 to find that the resultant vector has the components (4, 3).

Algebraically, we compute a vector sum by adding corresponding components:

$$\begin{aligned}\mathbf{c} &= \mathbf{a} + \mathbf{b} \\ &= (a_0, a_1) + (b_0, b_1) \\ &= (a_0 + b_0, a_1 + b_1)\end{aligned}$$

For the `Turtle`'s journey, that gives the following:

$$\begin{aligned}\mathbf{c} &= (1, 2) + (3, 1) \\ &= (1 + 3, 2 + 1) \\ &= (4, 3)\end{aligned}$$

The algebra matches the geometry! Now that we can add vectors, let's multiply them.

Scalar Multiplication

We can *scale* a vector by multiplying its components by a scalar. For example, given a vector $\mathbf{v} = (3, 4)$, we can scale its components by a factor of two like so:

$$\begin{aligned}2\mathbf{v} &= 2(3, 4) \\ &= (2 \cdot 3, 2 \cdot 4) \\ &= (6, 8)\end{aligned}$$

In this case, the vector's components grow larger, so its magnitude increases. Scaling can stretch, compress, or reverse a vector. Geometrically, we can view the different kinds of scaling as shown in Figure 5-10.

Starting from the original vector $\mathbf{v}$ in the top left, the vector is stretched by a factor of two in the top right, compressed by half in the bottom left, and finally reversed in the bottom right.

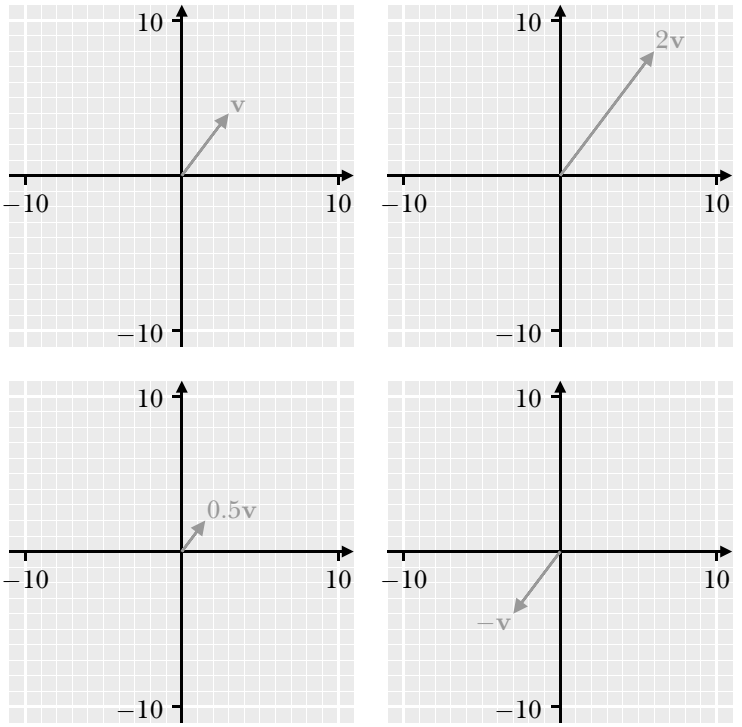


Figure 5-10: Different ways to scale a vector

We can combine vector operations to produce special vectors that are helpful for different purposes. For example, we can use a vector's magnitude $|\mathbf{v}|$ to scale it so that the resultant vector has a magnitude of 1:

$$\begin{aligned}\hat{\mathbf{v}} &= \frac{1}{|\mathbf{v}|}\mathbf{v} \\ &= \frac{1}{5}(3,\ 4) \\ &= \left(\frac{1}{5}\cdot 3,\ \frac{1}{5}\cdot 4\right) \\ &= (0.6,\ 0.8)\end{aligned}$$

The vector $\hat{\mathbf{v}}$ is known as a *unit vector*. Unit vectors show up all the time because it's useful to know in which direction things are changing, regardless of the magnitude of that change.

Vector Subtraction

We can subtract one vector from another by subtracting their corresponding components, like so:

$$\begin{aligned}\mathbf{c} &= \mathbf{a} - \mathbf{b} \\ &= (a_0, a_1) - (b_0, b_1) \\ &= (a_0 - b_0, a_1 - b_1)\end{aligned}$$

For example, consider the vectors $\mathbf{a} = (2, 4)$ and $\mathbf{b} = (6, 1)$. We can perform the subtraction $\mathbf{a} - \mathbf{b}$ algebraically:

$$\begin{aligned}\mathbf{c} &= \mathbf{a} - \mathbf{b} \\ &= (2, 4) - (6, 1) \\ &= (2 - 6, 4 - 1) \\ &= (-4, 3)\end{aligned}$$

We subtract the vectors' corresponding components to produce the resultant vector.

Vector subtraction turns out to be quite useful, too. Its power becomes clear once we visualize what's happening. Figure 5-11 demonstrates the process.

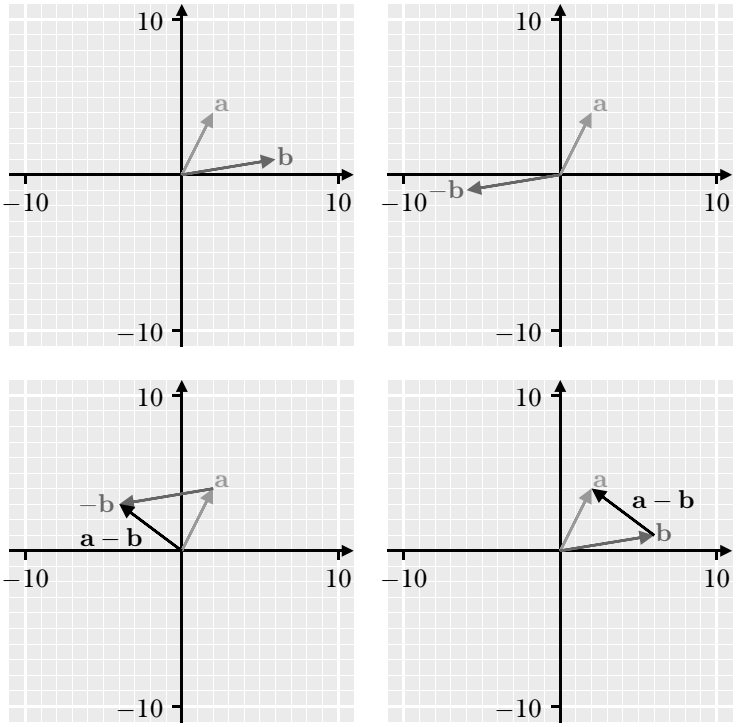


Figure 5-11: Vector subtraction

Geometrically, it's helpful to think of vector subtraction as a combination of negation and addition. Starting from the top left, the original vectors **a** and **b** appear in standard position. Next, the top-right figure shows **a** and **−b** drawn in standard position. The figure at the bottom left adds **a** and **−b**, producing the resultant vector **a** − **b**. Finally, the figure at the bottom right draws the original vectors **a** and **b** in standard position.

Notice that the operation **a** − **b** produces a vector that connects the tips of these vectors. Stated plainly, performing the vector subtraction **a** − **b** produces the vector that points from **b** to **a**, which is quite useful; we'll use vector subtraction to calculate forces in a moment.

Now that we've covered the basics of vector arithmetic, let's use that arithmetic to map between vectors.

Vector-Valued Functions

The functions we analyzed in Chapter 3, such as $f(x) = e^x$, are also known as *scalar-valued* functions because their codomains are sets of scalars. A *vector-valued* function is a function whose codomain is a set of vectors. Put differently, a vector-valued function maps an argument to a vector image. We'll start by focusing on vector-valued functions that map vector arguments to vector images. For example, the function $\mathbf{f}(\mathbf{x}) = 2\mathbf{x}$ scales its argument **x** by a factor of two. Given an argument such as $(3, 4)$, we can evaluate $\mathbf{f}(3, 4)$ as follows:

$$\begin{aligned}\mathbf{f}(3, 4) &= 2(3, 4) \\ &= (2 \cdot 3, 2 \cdot 4) \\ &= (6, 8)\end{aligned}$$

After substituting the argument $(3, 4)$ for **x**, we simply multiply each component by two to compute the function's image.

Now that you've got the basic idea, let's consider a classic example from physics that motivated Isaac Newton to develop his version of calculus. Newton sought to understand the motion of planets and bodies near Earth. After carefully analyzing experimental data, he found that he could describe planetary motion by assuming that objects with mass attract each other. The vector-valued function that describes this relationship looks like this:

$$\mathbf{F}(\mathbf{r}) = \frac{Gm_a m_b}{|\mathbf{r}|^2}\hat{\mathbf{r}}$$

This function has many terms, but don't let the algebra intimidate you. The quotient $\frac{Gm_a m_b}{|\mathbf{r}|^2}$ evaluates to a scalar, so the vector-valued function describing gravitation is just another scaling function, similar to $\mathbf{f}(\mathbf{x}) = 2\mathbf{x}$. Let's step through the terms in Newton's law of universal gravitation together:

- **F** is the force that describes how strongly body b pulls body a toward itself. Forces are measured in a unit called a Newton, N, which is equivalent to a $\text{kg} \cdot \text{m}/\text{s}^2$.

- $\mathbf{r}$ is the vector that extends from body a to body b. Its magnitude is measured in meters, m.
- G is Newton's universal gravitational constant. Its value is approximately $6.67 \times 10^{-11}\ \mathrm{N} \cdot \mathrm{m}^2/\mathrm{kg}$.
- m_a and m_b are the masses of the two bodies in kilograms, kg.
- $|\mathbf{r}|^2$ is the squared value of the magnitude of the distance vector r. It has a unit of square meters, m^2.
- $\hat{\mathbf{r}}$ is the unit vector pointing from object a to body b. It is unitless.

Figure 5-12 depicts a pair of bodies being drawn to each other by gravity.

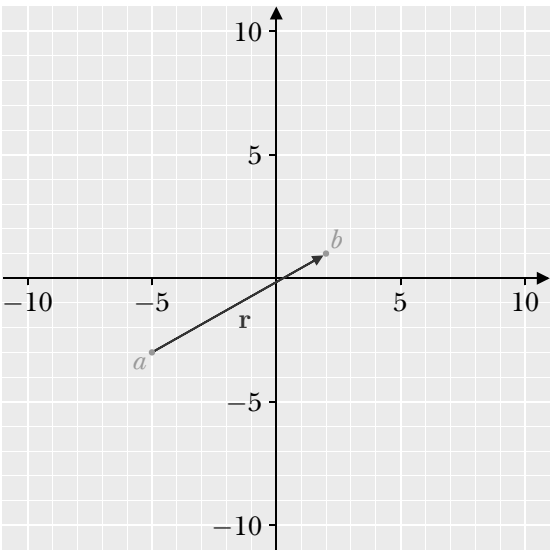


Figure 5-12: Setting up a gravity simulation

In this scenario, the vector $\mathbf{r} = (7, 4)$ connects body a to body b. Let's say each object has a mass of 10^6 kilograms, which is 1,000 metric tons. We can compute the strength of body b's pull on body a as follows:

$$\begin{aligned} F &= \frac{Gm_a m_b}{|\mathbf{r}|^2} \\ &= \frac{6.67 \cdot 10^{-11} \cdot 10^6 \cdot 10^6}{\sqrt{7^2 + 4^2}^2} \\ &= \frac{66.7}{65} \\ &\approx 1.03 \end{aligned}$$

The strength of the gravitational pull is about 1.03 N, which is about the same as the force you'd feel from a light breeze. Let's find the unit vector pointing from body a to body b:

$$\begin{aligned}\hat{\mathbf{r}} &= \frac{1}{|\mathbf{r}|}\mathbf{r} \\ &= \frac{1}{\sqrt{7^2 + 4^2}}(7,\ 4) \\ &\approx 0.124(7,\ 4) \\ &\approx (0.868,\ 0.496)\end{aligned}$$

Now that we have the unit vector and the magnitude of the force, let's compute the force vector:

$$\begin{aligned}\mathbf{F} &= |\mathbf{F}|\hat{\mathbf{r}} \\ &\approx 1.03(0.858,\ 0.496) \\ &\approx (0.884,\ 0.510)\end{aligned}$$

This force vector is relatively tiny, but it's enough to get the bodies moving.

Let's dig a bit deeper into the physics for a moment. Newton's third law of motion tells us that for every action, there's an equal and opposite reaction. When body b pulls on body a, body a pulls right back. We can restate this relationship in vector form. Given a force vector $\mathbf{F}_{ab}$ describing body b's pull on body a, we can say body a's pull on body b, $\mathbf{F}_{ba}$, is equal to $-\mathbf{F}_{ab}$.

Let's see Newton's third law in action. To do so, we'll need rewrite Newton's second law of motion in its vector form:

$$\ddot{\mathbf{x}} = \frac{1}{m}\mathbf{F}$$

Given a body's mass and a force vector, we can compute its acceleration vector. Let's see how the force of gravity would accelerate body a with its mass m_a:

$$\begin{aligned}\ddot{\mathbf{x}}_a &= \frac{1}{m_a}\mathbf{F}_{ab} \\ &= \frac{1}{m_a}\frac{Gm_am_b}{|\mathbf{r}_{ab}|^2}\hat{\mathbf{r}}_{ab} \\ &= \frac{Gm_b}{|\mathbf{r}_{ab}|^2}\hat{\mathbf{r}}_{ab}\end{aligned}$$

The first equation replaces the force vector $\mathbf{F}$ with Newton's universal law of gravitation. Note that it now specifies that the vector $\mathbf{r}_{ab}$ points from body a to body b. The second equation then simplifies the quotient by canceling out m_a, leaving us with the relationship between body a's acceleration

and its distance from body b. We could write the second-order ordinary differential equation for body b in a similar way:

$$\ddot{\mathbf{x}}_b = \frac{Gm_a}{|\mathbf{r}_{ba}|^2}\hat{\mathbf{r}}_{ba}$$

The only differences on the right-hand side of the equation are the mass m_a and the vector $\hat{\mathbf{r}}_{ba}$, which points from body b to body a. Now that we've calculated second-order equations, we can solve them using Euler's method.

NumPy Arrays

In this section, we'll begin using the NumPy package, which provides a powerful object for performing vector arithmetic. NumPy is used heavily in scientific computing and data science because it makes it easy to do many calculations quickly. It will become a workhorse for us throughout the rest of the book.

Creating ndarray Objects

NumPy's `ndarray` object is like a high-performance version of a list. Like lists, we can access the elements of an `ndarray` using indices and square brackets. However, the length of `ndarrays` are fixed. Also, unlike lists, the elements of an `ndarray` must all have the same data type.

We'll use `ndarray` objects to represent N-dimensional vectors. To begin, open your notebook and import the NumPy package:

```
import numpy as np
```

It's common practice to give NumPy the shorthand `np`. Now that we've imported the package, let's create an `ndarray` containing a few numbers:

```
np.array([8, 6, 7, 5, 3, 0, 9])
-------------------------------
array([8, 6, 7, 5, 3, 0, 9])
```

The `array` function accepts a sequence as its argument, such as a list or another `ndarray`. We'll follow convention by using square brackets (`[]`) to pass a *list* as an argument.

It's easy to accidentally pass individual numbers to `array`, which causes an error:

```
np.array(8, 6, 7, 5, 3, 0, 9)
-----------------------------
TypeError                                 Traceback (most recent call last)
Cell In[6], line 1
----> 1 np.array(8, 6, 7, 5, 3, 0, 9)

TypeError: array() takes from 1 to 2 positional arguments but 7 were given
```

Let's create an `ndarray` variable and begin inspecting it:

```
x = np.array([8, 6, 7, 5, 3, 0, 9])
x
-
array([8, 6, 7, 5, 3, 0, 9])
```

We can access individual array elements using the same syntax as for lists:

```
x[0]
----
np.int32(8)
```

The number's data type is printed along with its value. In my web browser, the version of NumPy I used while writing this book defaults to representing numbers with an integer that uses 32 bits of memory. Let's inspect the last element in the `ndarray`, as well:

```
x[-1]
-----
np.int32(9)
```

As expected, the last element is also a 32-bit integer.

As with lists, we can reassign an `ndarray`'s elements:

```
x[0] = 2
x
-
array([2, 6, 7, 5, 3, 0, 9])
```

The zeroth element changed from an 8 to a 2.

One `ndarray` feature that we'll make great use of in the next chapter is the ability to *slice* its values. For example, we could reassign the first three elements. Run the following statements in separate code cells:

```
x[:3]
-----
array([2, 6, 7])

x[:3] = 1
x
-
array([1, 1, 1, 5, 3, 0, 9])
```

The syntax for slicing gives us the option to specify a start index, a stop index, and a stride. Let's see this syntax in action quickly:

```
print(x[:])   # All elements
print(x[-2:]) # Last two elements
print(x[4:6]) # Elements 4 and 5
print(x[::2]) # Every other element
```

```
----------------------------------
[1 1 1 5 3 0 9]
[0 9]
[3 0]
[1 1 3 9]
```

There are many ways to create an `ndarray`, depending on our needs. For example, if we want to use floating-point numbers instead of integers, we can either pass in a decimal number or specify the type explicitly:

```
x = np.array([8.0, 6, 7, 5, 3, 0, 9])
x
-
array([8., 6., 7., 5., 3., 0., 9.])

x = np.array([8, 6, 7, 5, 3, 0, 9], dtype=np.float64)
x
-
array([8., 6., 7., 5., 3., 0., 9.])
```

In the first example, the zeroth element is a floating-point number `8.0`, so the entire `ndarray` contains floats. In the second example, the optional `dtype` argument is set to the `float64` type, which uses 64 bits of memory.

We'll often need a sequence of numbers, say from 0 to 10, which is easy to create using the `arange` function:

```
x = np.arange(10)
x
-
array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
```

The `arange` function works similarly to `range` in that we can specify the start, stop, and step values as needed:

```
x = np.arange(0, 20, 5)
x
-
array([0, 5, 10, 15])
```

Soon, we'll also need to create a sequence of evenly spaced numbers in an interval, which is easy to do with the `linspace` function:

```
x = np.linspace(0, 1, 5)
x
-
array([0., 0.25, 0.5, 0.75, 1])
```

Sometimes, it's nice to just have an `ndarray` full of zeros or ones that we can reassign as needed later:

```
x = np.zeros(5)
x
-
array([0., 0., 0., 0., 0.])

x = np.ones(5)
x
-
array([1., 1., 1., 1., 1.])
```

Finally, we can create an `ndarray` filled with random floating-point numbers in the interval [low, high) using the `random.uniform` function:

```
x = np.random.uniform(0, 1, 5)
x
-
array([0.92498088, 0.13646163, 0.35973786, 0.78196611, 0.94519511])
```

The first two arguments specify the low and high values of the interval from which we're drawing random numbers. The last argument is the shape of the `ndarray` to create. The numbers you see will be different each time you run the code, but will always fall within the given interval.

Now that we've seen a few ways to create an `ndarray`, let's put them to use.

Performing Operations and Broadcasting

The `ndarray` object supports all the vector arithmetic operations we've covered thus far. Let's begin with scalar multiplication:

```
x = np.arange(5)
x
-
array([0, 1, 2, 3, 4])

2 * x
-----
array([0, 2, 4, 6, 8])

x
-
array([0, 1, 2, 3, 4])
```

After creating an `ndarray` with the numbers 0 through 4, we multiply its elements by two to produce a new `ndarray`. Note that the original `ndarray` is unchanged.

The ability to apply an operation to an entire `ndarray` is called *broadcasting*. The rest of the standard arithmetic operators also broadcast:

```
x / 2
-----
```

```
array([0. , 0.5, 1. , 1.5, 2. ])

x ** 2
------
array([0, 1, 4, 9, 16])

x % 2
-----
array([0, 1, 0, 1, 0])
```

Division, exponentiation, and modular division are applied to each element in the `ndarray`, also without modifying the original.

Now that we've covered scalar arithmetic, let's see those new vector operations in action. Given a pair of `ndarray` objects, we can scale and add them like so:

```
a = np.array([1, 2])
b = np.array([3, 4])
a + b
-----
array([4, 6])

5 * a + b
---------
array([8, 14])

5 * a + 6 * b
-------------
array([23, 24])
```

Once we create the `ndarrays`, we can apply a mix of addition and scalar multiplication to produce a new `ndarray` that's a combination of the originals.

Like lists, we can also unpack the elements of an `ndarray`:

```
one, two = a
print(one)
print(two)
----------
1
2
```

NumPy also has many helpful functions for computing with `ndarray` objects, so let's explore a few of them.

Calling Universal Functions

NumPy provides many functions, called *universal functions*, that can help you to perform common calculations. They accept one or more `ndarray` objects

as arguments and usually return a number or another ndarray. Let's create an ndarray that we'll use to test them out:

```
x = np.arange(5)
x
-
array([0, 1, 2, 3, 4])
```

Now let's quickly review a few useful universal functions:

```
print(np.sum(x))
print(np.max(x))
print(np.min(x))
print(np.clip(x, 0, 2))
print(np.sin(x))
print(np.cos(x))
print(np.exp(x))
----------------
10
4
0
[0 1 2 2 2]
[ 0.         0.84147098  0.90929743  0.14112001 -0.7568025 ]
[ 1.         0.54030231 -0.41614684 -0.9899925  -0.65364362]
[ 1.         2.71828183  7.3890561  20.08553692 54.59815003]
```

The first function, sum, adds up the elements in an ndarray and returns the total value. Next, max and min return the maximum and minimum values in the ndarray, respectively. Then, we have clip, which limits the values in an ndarray to a given interval, in this case [0, 2]. Finally, sin, cos, and exp compute the values of the respective functions for each element in the ndarray.

The last function we'll cover can compute the magnitude of a vector:

```
b = np.array([3, 4])
mag = np.linalg.norm(b)
print(mag)
----------
5.0
```

In this case, the linalg.norm function works by applying Pythagoras's theorem behind the scenes. linalg.norm supports different definitions of magnitude, but we won't cover them here.

There are many more universal functions available, and you can visit the NumPy website to read their documentation. Now that you have a handle on working with ndarrays, you're ready to use them with Turtles.

Programming with Vectors

Let's see how to apply vector arithmetic to simulate motion. Enter *chapter_05.ipynb* and enter the following in the next available code cell:

```
from ipycc.turtle import Turtle
import calcplotlib as plt
```

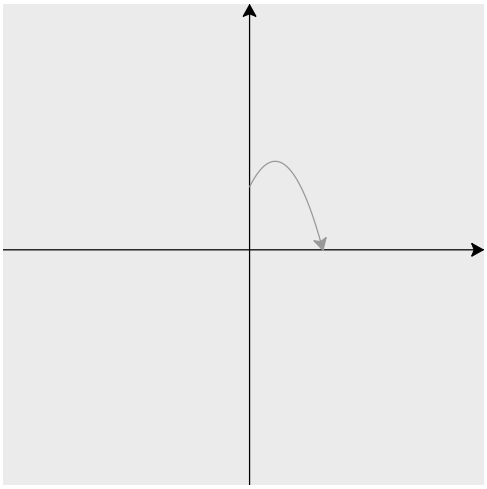
In this section, we'll use `ndarrays` to move a `Turtle` around the screen. Then, we'll visualize vectors with `calcplotlib`.

Simulating Surface Gravity

We can estimate gravitational acceleration near the surface of the earth as the vector $\mathbf{a} = (0, -9.8)$, which says that gravity provides a steady pull downward toward the surface. Let's simulate this model by watching a `Turtle` mimic a hopping chicken:

```
# Show the plot
plt.show()

# Prepare to draw
plt.clear()
plt.draw_axes()

# Set the initial values
acc = np.array([0.0, -9.8])
vel = np.array([10.0, 20.0])
pos = np.array([0.0, 50.0])

# Configure the Turtle
t = Turtle()
t.color(plt.GRAY60)
x, y = pos
t.teleport(x, y)

# Configure the time step
dt = 0.001

# Run Euler integration
while t.ycor() > 0:
    # Update velocity
    vel += dt * acc
    # Update position
    pos += dt * vel
    # Set heading toward next position
    x, y = pos
    angle = t.towards(x, y)
    t.seth(angle)
```

```
    # Move the Turtle
    t.goto(pos)
```



We begin by showing the plot, clearing it, and drawing our axes. Next, we set the initial conditions for applying Euler's method, configure a light gray `Turtle`, and set the time step. Note that we're creating all `ndarrays` using `float` values. The `teleport` method won't accept an `ndarray` as an argument, so we unpack the elements of `pos` and pass them individually.

I chose to use a `while` loop so that the simulation runs until the `Turtle` reaches the ground. Within the loop, we begin by applying Euler's method to update the `vel` and `pos` vectors. Next, we compute the heading the `Turtle` must follow to reach its next position and point it in that direction. Like `teleport`, we have to unpack the elements of `pos` and pass them individually to `towards`. Finally, we watch the hopping `Turtle` glide through the digital air. Notice the path is a parabola, just like we'd expect near Earth's surface. Fun!

Now that we've seen how gravity affects one `Turtle`, let's see what happens when massive `Turtles` attract each other.

Simulating Gravity Between Turtles

Newton's universal law of gravitation is a useful model, so let's define a vector-valued function to apply it:

```
def apply_gravity(r, mass_a, mass_b):
    """Computes the force of gravity between two bodies"""
    G = 6.67 * 10 ** (-11)
    r_mag = np.linalg.norm(r)
    strength = G * mass_a * mass_b / r_mag ** 2
    unit_vector = (1 / r_mag) * r
    return strength * unit_vector
```

The `apply_gravity` function has three parameters to describe the gravitational system under study. The first parameter, `r`, is the `ndarray` extending from body *a* to body *b*. The next two parameters, `mass_a` and `mass_b`, are the masses of bodies *a* and *b*. Let's see if the function returns the same result we computed by hand earlier in the chapter:

```
r = np.array([7.0, 4.0])
mass_a = 10 ** 6
mass_b = 10 ** 6
F = apply_gravity(r, mass_a, mass_b)
mag = np.linalg.norm(F)
print(f"The force vector {F} has a magnitude of {mag}")
```

```
The force vector [0.89095104 0.50911488] has a magnitude of 1.0261538461538462
```

The calculation checks out! Now, let's see how the force vector changes when we consider bodies that are farther apart:

```
# Create the position vectors for bodies a and b
pos_a = np.array([-50.0, -30.0])
pos_b = np.array([20.0, 10.0])

# Compute the force vector
r = pos_b - pos_a
mass_a = 10 ** 6
mass_b = 10 ** 6
F = apply_gravity(r, mass_a, mass_b)
F
```

```
array([0.00890951, 0.00509115])
```

The components are tiny! That's because Newton's universal law of gravitation is an inverse-square law, meaning its strength is inversely proportional to the square of the distance:

$$F \propto \frac{1}{r^2}$$

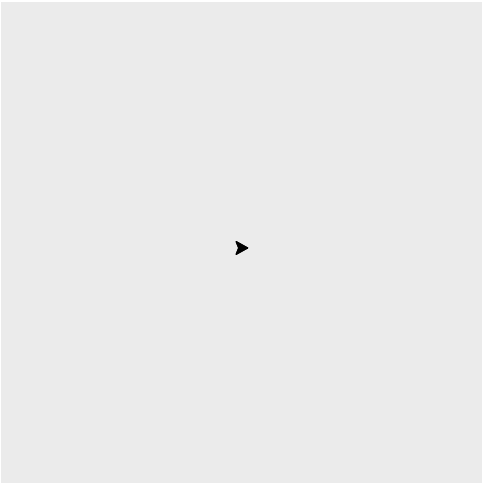
When bodies move farther apart, the gravitational force between them weakens very quickly. You'll observe this effect for yourself in a moment.

Let's take Newton's law of universal gravitation for a spin by simulating the motion of multiple bodies and visualizing their journey. Fortunately for us, it's easy to create multiple `Turtle`s that can follow their own paths. Run the following commands in a code cell to get started:

```
plt.clear()
t.clear()
t.hideturtle()
```

Now let's create a pair of new `Turtles` like so:

```
# Show the plot
plt.show()

# Create new Turtles
body_a = Turtle()
body_b = Turtle()
```



The variables `body_a` and `body_b` each reference a unique `Turtle` that we can give a mass and velocity. We'll use these `Turtles`, both shown on top of each other, to integrate Newton's laws forward in time with Euler's method. Let's begin by configuring the simulation:

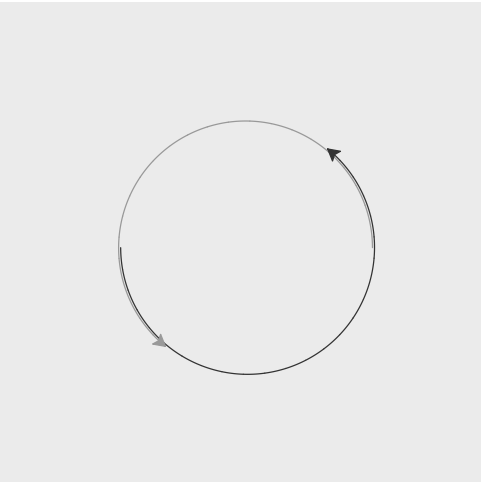
```
# Set the initial values
t_0 = 0
pos_a = np.array([100.0, 0.0])
pos_b = np.array([-100.0, 0.0])
vel_a = np.array([0.0, 410.0])
vel_b = np.array([0.0, -410.0])

# Configure the Turtles
body_a.color(plt.GRAY60)
body_b.color(plt.GRAY20)

# Set the parameters
mass_a = 10 ** 18
mass_b = 10 ** 18

# Configure the time step
t_f = 1
```

```
dt = 0.001
num_steps = int(t_f / dt)
```

First, we set the initial values for time, position, and velocity. Time is a scalar, while the position and velocity for each body are `ndarrays`. Next, we prepare our `Turtle` objects for drawing by setting their colors. After that, we set the (mathematical) parameters for the differential equations, namely mass. The next few statements organize the time stepping.

Finally, it's time for simulation. Run the following code cell to set the `Turtles` in motion:

```
# Show the plot
plt.show()

# Run Euler integration
x, y = pos_a
body_a.teleport(x, y)
x, y = pos_b
body_b.teleport(x, y)

for i in range(1, num_steps):
    # Compute the force of gravity
    r_ab = pos_b - pos_a
    force_ab = apply_gravity(r_ab, mass_a, mass_b)
    force_ba = -force_ab
    # Compute the acceleration
    acc_a = (1 / mass_a) * force_ab
    acc_b = (1 / mass_b) * force_ba
    # Update velocity
    vel_a += dt * acc_a
    vel_b += dt * acc_b
    # Update position
    pos_a += dt * vel_a
    pos_b += dt * vel_b
    # Set heading toward the next position
    x, y = pos_a
    angle = body_a.towards(x, y)
    body_a.seth(angle)
    x, y = pos_b
    angle = body_b.towards(x, y)
    body_b.seth(angle)
    # Move the Turtles
    body_a.goto(pos_a)
    body_b.goto(pos_b)
```



After setting all the current values to their initial values, we move the `Turtle`s to their starting positions and prepare to track their motion as we integrate. Note the `teleport` and `towards` methods don't accept `ndarrays` as arguments, so we have to unpack their elements and pass them individually.

Within the loop, we begin each time step by moving the `Turtle` objects to their current position. Next, we compute the gravitational attraction between the bodies using a little vector arithmetic and the `apply_gravity` function. After that, we update each body's acceleration, velocity, and position vectors.

Newton's law of universal gravitation predicts that a pair of planets initially moving in opposite directions will wind up orbiting each other. But what happens when we add another body? In a word, chaos.

Observing the Chaos of the n-Body Problem

One of the first problems Isaac Newton tried to solve with calculus was predicting the future positions of the sun, earth, and moon. This problem is commonly known as a *three-body problem* or, more generally, an n-*body problem*. Newton failed to find a solution in his own time, and mathematicians in later generations discovered that it was impossible to do so, aside from a few special cases. Thankfully, numerical solutions have proven accurate enough to carry humanity into outer space.

We can study the *n*-body problem for ourselves by adding another body to the gravity simulation. Add the following code cell to your notebook:

```
# Create a new Turtle
body_c = Turtle()
```

Now add another code cell and simulate gravitation between the three `Turtles`:

```
# Reset the Turtles
body_a.reset()
body_b.reset()
body_c.reset()

# Set the initial values
t_0 = 0
pos_a = np.array([100.0, 0.0])
pos_b = np.array([-100.0, 0.0])
pos_c = np.array([1.0, 0.0])
vel_a = np.array([0.0, 500.0])
vel_b = np.array([0.0, -500.0])
vel_c = np.array([0.0, 0.0])

# Configure the Turtles
body_a.color(plt.GRAY60)
body_b.color(plt.GRAY20)
body_c.color(plt.WHITE)

# Set the parameters
mass_a = 10 ** 18
mass_b = 10 ** 18
mass_c = 10 ** 17

# Configure the time step
t_f = 1
dt = 0.001
num_steps = int(t_f / dt)
```

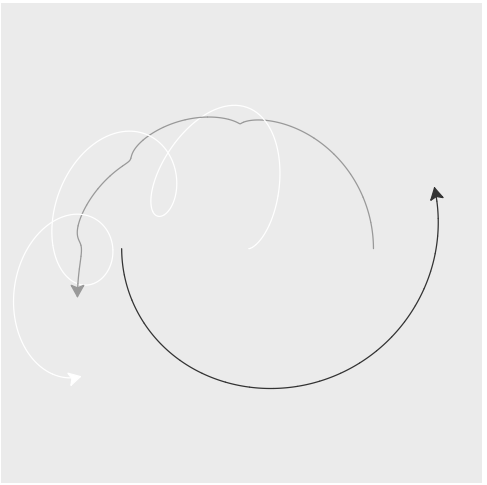
The first code cell adds another `Turtle` object named `body_c`, while the second contains the extended simulation. We need to make several additions to the code, so let's quickly review them.

First, we give `body_c` an initial position and velocity. Next, we ensure that body *c* appears as a white circle and prepare to move it into its initial position. After that, we give body *c* mass before setting its position and velocity for the first time step.

Now let's run Euler integration for one second:

```
# Show the plot
plt.show()

# Run Euler integration
body_a.teleport(pos_a)
body_b.teleport(pos_b)
body_c.teleport(pos_c)
```

```
for i in range(1, num_steps):
    # Compute the force of gravity
    r_ab = pos_b - pos_a
    r_ac = pos_c - pos_a
    r_bc = pos_c - pos_b
    force_ab = apply_gravity(r_ab, mass_a, mass_b)
    force_ba = -force_ab
    force_ac = apply_gravity(r_ac, mass_a, mass_c)
    force_ca = -force_ac
    force_bc = apply_gravity(r_bc, mass_b, mass_c)
    force_cb = -force_bc
    # Compute the acceleration
    acc_a = (1 / mass_a) * (force_ab + force_ac)
    acc_b = (1 / mass_b) * (force_ba + force_bc)
    acc_c = (1 / mass_c) * (force_ca + force_cb)
    # Update velocity
    vel_a += dt * acc_a
    vel_b += dt * acc_b
    vel_c += dt * acc_c
    # Update position
    pos_a += dt * vel_a
    pos_b += dt * vel_b
    pos_c += dt * vel_c
    # Set heading toward the next position
    x, y = pos_a
    angle = body_a.towards(x, y)
    body_a.seth(angle)
    x, y = pos_b
    angle = body_b.towards(x, y)
    body_b.seth(angle)
    x, y = pos_c
    angle = body_c.towards(x, y)
    body_c.seth(angle)
    # Move the Turtles
    body_a.goto(pos_a)
    body_b.goto(pos_b)
    body_c.goto(pos_c)
```

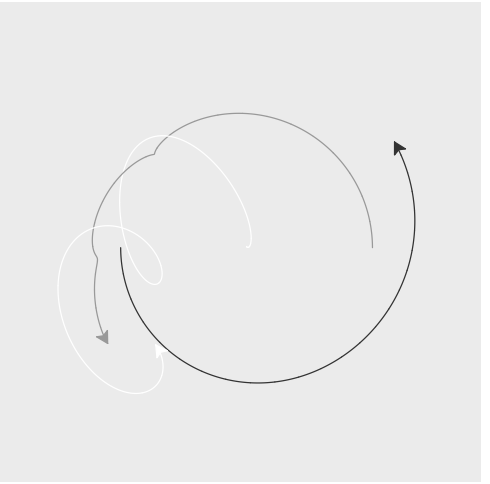


The computations within the `for` loop expand a bit to account for the interactions between each pair of bodies. After computing all the forces and accelerations, we update each body's velocity and position. This code works, but it already seems a bit repetitive for $n = 3$ bodies. You'll tidy up this simulation in an exercise at the end of the chapter so that you can simulate any number of bodies.

One of the most interesting features of the *n*-body problem is that its behavior is *chaotic*. In mathematics, chaotic behavior doesn't imply randomness. Far from it. Chaotic systems exhibit *determinism*, meaning that their current state determines their next state. In our *n*-body example, Newton's laws tell us exactly where each body will move next. Chaotic behavior also shows *irregularity*, meaning the system never returns to the exact same conditions. If it did, then determinism would cause it to follow its old path in a loop. The final, and perhaps the most unsettling, feature is that chaotic systems show *sensitive dependence on initial conditions*. A tiny change can result in a wildly different outcome.

Let's get our first glimpse of chaos in action by slightly tweaking the initial position of body *c*. Make the following change to `pos_c` and re-run the simulation:

```
--snip--

# Set the initial values
t_0 = 0
pos_a = np.array([100.0, 0.0])
pos_b = np.array([-100.0, 0.0])
pos_c = np.array([0.0, 1.0]) # A little bit up

--snip--
```



Moving body c's initial position a little bit upward caused its final position to change dramatically. If we were modeling the sun-earth-moon system, the moon would be in a completely different phase!

What You Learned

In this chapter, you formalized differential equations and put them to use simulating the universe. First, you defined ordinary differential equations along with their explicit and numerical solutions. Next, you learned the basics of vector arithmetic. You then used vectors to study differential equations that involve motion, which ultimately led to chaos. In the next chapter, you'll develop a more holistic view of differential equations.

Exercises

Use these exercises to practice the concepts you've learned in this chapter, including explicit and numerical methods for solving differential equations and vector arithmetic. The first several exercises focus on explicit solutions to differential equations using familiar examples. Then you'll analyze the complexity of vector arithmetic and compare the performance of NumPy to your own implementation. The final batch of exercises gives you a chance to apply Euler's method to simulate forces, populations, atmospheric chemicals, and fireflies.

Solving Differential Equations

5-1. Review the compound interest formula from Chapter 1. Then write a differential equation to model an investment that earns compound interest and solve it explicitly.

5-2. Find the explicit solution to the first-order ordinary differential equation $\dot{x} = v_0$, which describes a body moving with constant velocity.

5-3. Find the explicit solution to the second-order ordinary differential equation $\ddot{x} = a_0$, which describes a body moving with constant acceleration. Note: You'll have to integrate twice.

5-4. Given that $e^{\ln(x)} = x$ and $(a^m)^n = a^{mn}$, explain why we were able to transform our germinator robot population model from $g(t) = g_0 2^t$ to $g(t) = g_0 e^{\ln(2)t}$.

Analyzing Vector Operations

5-5. Determine the complexity of vector addition and scalar multiplication.

5-6. Implement vector addition and scalar multiplication using lists. Then use the `%timeit` magic command to measure the performance of your implementation and compare it to the performance of NumPy.

Simulating Forces with Vectors

5-7. Simulate a ball bouncing off the edges of the screen using `ndarray` objects and `if` statements.

5-8. Add friction to your bouncing simulation based on the vector-valued function $\mathbf{F}(\mathbf{v}) = -c_f\mathbf{v}$. The parameter c_f is known as the *coefficient of friction*. Set its value to 0.001 to get started.

5-9. Add a second ball to your bouncing simulation. Bonus points if you make them bounce off of each other.

5-10. Rewrite the n-body simulation so that it can easily handle $n > 2$ bodies. One approach is to use lists and a nested loop within the time loop. Test your implementation by comparing the output to the 3-body simulation from the end of the chapter.

Simulating Populations

5-11. Extend the germinator robot population model to include a death rate that is also proportional to the current robot population, as in the following differential equation:

$$\begin{aligned}\dot{g} &= k_B g - k_D g \\ &= (k_B - k_D)g\end{aligned}$$

Use Python to explore how the robot population changes in each of the following scenarios:

- $k_B > k_D$
- $k_B = k_D$
- $k_B < k_D$

Now, describe the relationship between a population's birth rate and death rate in simple terms.

5-12. Population models often include a *carrying capacity* to represent environmental factors that limit the size of the population. For example,

let's assume that germinator robots travel more slowly than the speed of light, so they'll only be able to use resources within a finite region of the universe. That region may only have enough matter and energy to support N robots. We can model this scenario using the following differential equation:

$$\dot{g} = (k_B - k_D)(1 - \frac{g}{N})g$$

The term $(1 - \frac{g}{N})$ accounts for the carrying capacity of the environment. Explain what happens to $\dot{g}$ in the following situations:

- $g > N$
- $g = N$
- $g < N$

Now, use Python to simulate different scenarios for robot populations that include birth, death, and carrying capacity.

5-13. Germinator robots are known to catch viruses from time to time. Read the Wikipedia page about *compartmental models* through the section on the susceptible, infectious, and recovered (SIR) model. Then apply the following SIR model to simulate an outbreak of robot flu in a population of 1,000 robots:

$$S_0 = 997 \qquad \beta = 0.4$$
$$I_0 = 3 \qquad \gamma = 0.04$$
$$R_0 = 0$$

$$\dot{S} = -\beta SI$$
$$\dot{I} = \beta SI - \gamma I$$
$$\dot{R} = \gamma I$$

The time derivatives have units of robots/day. These coupled ordinary differential equations are *nonlinear* because the state variables are multiplied, as in SI. Run your simulation for one month and print the results.

Simulating Atmospheric Chemistry

The atmosphere is a molecular soup that constantly changes, and you can model that change with differential equations. The bottom 100 km of the atmosphere, called the *homosphere*, is about 78% molecular nitrogen, N_2, and 21% molecular oxygen, O_2. The remaining 1% of the atmosphere contains many *variable gases* that have a large impact on its state.

Elementary chemical reactions have the form

$$\text{reactants} \rightarrow \text{products}$$

Reactions describe how input molecules, or *reactants*, transform into output molecules, or *products*. A *reaction rate* specifies how quickly

a reaction occurs. You're going to analyze a few important chemical reactions that lead to the production of ozone gas, O_3. Earth's natural ozone layer is about 25 km above the surface and protects life from harmful radiation. At the surface, ozone production causes a range of health problems.

Instead of modeling individual molecules, you'll solve differential equations describing their *concentration* expressed as the number of molecules per cubic centimeter. You'll write the concentration of molecules and their time derivatives as follows:

$$[O_3] \text{ molec/cm}^3$$

$$\frac{d[O_3]}{dt} \text{ molec/(cm}^3 \cdot \text{s)}$$

5-14. A *photolysis reaction* breaks a single reactant into multiple products. Photolysis occurs when a particle of light, or *photon*, with enough energy hits a molecule. An important photolysis reaction for ozone formation is:

$$NO_2 + h\nu \rightarrow NO + O$$

Nitrogen dioxide, NO_2, produces nitrogen monoxide, NO, and atomic oxygen, O. You can model this reaction using a parameter called its *rate coefficient*, the concentrations of relevant chemicals, and an initial condition:

$$k = 8.82 \times 10^{-3} \text{ s}^{-1}$$

$$[NO_2] = 10^{10} \text{ molec/cm}^3$$

$$\frac{d[NO]}{dt} = k[NO_2]$$

Solve this differential equation using separation of variables. Assume $[NO_2]$ stays constant, so the variables are [NO] and t.

5-15. A *kinetic reaction* occurs when multiple reactants collide and transform into products. An important kinetic reaction for ozone formation is:

$$O + O_2 \rightarrow O_3$$

Atomic oxygen, O, and molecular oxygen, O_2, combine to produce ozone, O_3. You can also model this reaction using its rate coefficient and initial conditions:

$$k = 6.59 \times 10^{-34} \text{ cm}^3/\text{s}$$

$$[O] = 10 \text{ molec/cm}^3$$

$$[O_2] = 5.34 \times 10^{18} \text{ molec/cm}^3$$

$$\frac{d[O_3]}{dt} = k[O][O_2]$$

In this equation, you multiply the rate coefficient by the concentration of both reactants. Solve this differential equation using separation of variables. Assume [O] and $[O_2]$ stay constant, so the variables are $[O_3]$ and t.

5-16. In cities, NO reacts with a class of chemicals called *reactive organic gases*, RO_2, to produce ozone. The reactions are:

$$\begin{aligned} NO + RO_2 &\rightarrow NO_2 + RO & k_a &= 7.7 \times 10^{12}\ cm^3/s \\ NO_2 + h\nu &\rightarrow NO + O & k_b &= 8.82 \times 10^{-3}\ s^{-1} \\ O + O_2 &\rightarrow O_3 & k_c &= 6.59 \times 10^{-34}\ cm^3/s \end{aligned}$$

Write differential equations to model the production of NO_2, O, and O_3:

$$\frac{d[NO_2]}{dt} =$$
$$\frac{d[O]}{dt} =$$
$$\frac{d[O_3]}{dt} =$$

5-17. Use Euler's method to simulate the production of O_3 for one second using the following initial concentrations in molec/cm^3:

$$[O] = 0 \quad [O_2] = 5.34 \times 10^{18} \quad [O_3] = 0$$

$$[NO] = 10^{12} \quad [NO_2] = 10^{10}$$

$$[RO] = 0 \quad [RO_2] = 10^{11}$$

Assume $[O_2]$ and $[RO_2]$ are constant. Use a time step of 0.0001 s and run the simulation for 1 second.

Simulating Sync

Now that you've simulated chaos, let's head in the opposite direction to study *synchronization*, or *sync*, which is the tendency for coordinated behavior. For example, we witness sync when fireflies flash in unison or dancers move as one. As a first step, you will simulate a pair of *oscillators* who change their states according to a pair of coupled differential equations:

$$\dot{\theta}_a = \omega_a + \frac{K}{2}\sin(\theta_b - \theta_a)$$
$$\dot{\theta}_b = \omega_b + \frac{K}{2}\sin(\theta_a - \theta_b)$$

The variable θ is an oscillator's *phase*, which changes over time. For example, the brightness of a firefly oscillates between dark and light values as its phase changes. The parameters in this model are ω, the *natural frequency* at which an oscillator tends to oscillate, and K, the strength of the connection, or *coupling*, between the oscillators. Put simply, each oscillator changes according to its own internal rhythm while also coordinating with its peers. All together, this system of coupled differential equations is known as the Kuramoto model.

5-18. Apply Euler's method with a time step of `0.001` to simulate a pair of `Turtle` objects that sync their heading for two seconds. Use the following initial values and parameters to get started:

Initial values: $\theta_a = 0$, $\theta_b = 1.5$
Parameters: $\omega_a = 5$, $\omega_b = 8$, $K = 1$

Experiment by adjusting the initial values and parameters, starting with K.

5-19. What values of K cause the oscillators with $\omega_a = 2$ and $\omega_b = 5$ to synchronize their phases?

5-20. What does the expression $\sin(\theta_b - \theta_a)$ represent? Start by considering the case $\theta_a \approx \theta_b$.

5-21. What happens when an oscillator's natural frequency is much greater than the coupling strength, as in $\omega_a \gg K$? Try to explain this behavior based on the differential equations above.

5-22. The general version of the Kuramoto model for N oscillators is:

$$\dot{\theta}_i = \omega_i + \frac{K}{N}\sum_{j=0}^{N-1}\sin(\theta_j - \theta_i)$$

Rewrite your sync simulation to handle $N = 10$ oscillators. Use lists and `ndarrays` where appropriate. Give each oscillator a random natural frequency $5 \leq w < 10$ and initial phase $0 \leq \theta < 2\pi$.

5-23. What values of K cause the larger set of oscillators to synchronize their phases?